GraphHunter

Especificación

Lucas Sotomayor
Base4Security

13 de mayo de 2026

Índice general

1. Introducción **1**
- 1.1. Para qué sirve esta especificación . . . 1
- 1.2. Qué no está aquí . . . 1
- 1.3. Estabilidad y versionado . . . 1
- 1.4. Convenciones del libro . . . 2
- 1.5. Mapa del libro . . . 2

2. HTTP API canónica **3**
- 2.1. Transporte y autenticación . . . 3
- 2.2. Esquema introspectivo . . . 3
- 2.3. Salud y telemetría . . . 3
- 2.4. Catálogo y schema . . . 3
- 2.5. Navegación del grafo . . . 4
- 2.6. Caza (hunts) . . . 4
- 2.7. Ingesta . . . 4
- 2.8. Anotación . . . 5
- 2.9. Anomaly y enrichment . . . 5
- 2.10. Exportación . . . 5
- 2.11. Invariantes . . . 5
- 2.12. Agentic (slow lane) . . . 5
- 2.13. Dead Letter Queue . . . 6
- 2.14. SIEM (Sentinel) . . . 6
- 2.15. Datasets y utilidad . . . 6
- 2.16. Códigos de respuesta . . . 6
- 2.17. Total . . . 6

3. Herramientas MCP **7**
- 3.1. Protocolo . . . 7
- 3.2. Dominio `agentic` (9 tools) . . . 7
- 3.3. Dominio `catalogs` (4 tools) . . . 8
- 3.4. Dominio `data-quality` (7 tools) . . . 8
- 3.5. Dominio `export` (3 tools) . . . 8
- 3.6. Dominio `graph-views` (4 tools) . . . 8
- 3.7. Dominio `hunting` (6 tools) . . . 8
- 3.8. Dominio `integrations` (3 tools) . . . 9
- 3.9. Dominio `navigation` (10 tools) . . . 9
- 3.10. Dominio `notes` (6 tools) . . . 9
- 3.11. Dominio `scoring` (3 tools) . . . 9
- 3.12. Contrato común . . . 10
- 3.13. Tests contractuales . . . 10

4. Lenguaje de hipótesis (DSL) 11
4.1. Propósito . . . 11
4.2. Gramática v1 (estable) . . . 11
4.3. Entity types built-in . . . 12
4.4. Relation types built-in . . . 12
4.5. Predicados built-in . . . 12
4.6. Ejemplos . . . 12
4.7. Extensibilidad (ADR-005) . . . 13
4.8. Compatibilidad . . . 13

5. Formatos de log soportados 14
5.1. El trait `LogParser` . . . 14
5.2. Sysmon (Windows Event Log XML) . . . 14
5.3. Sentinel / Log Analytics (JSON) . . . 15
5.4. CSV genérico . . . 15
5.5. Generic (heurístico) . . . 15
5.6. FortiAnalyzer . . . 15
5.7. AWS Cognito . . . 15
5.8. IIS (W3C Extended Log) . . . 16
5.9. Tres carriles de ingesta . . . 16
5.9.1. Fast lane — VRL compilado . . . 16
5.9.2. Slow lane — LLM-as-compiler . . . 16
5.9.3. Learning lane — drift-as-signal . . . 17
5.10. Canonical OCSF . . . 17
5.11. FieldConfig como contrato . . . 17
5.12. Cómo agregar un parser . . . 18

6. Superficie pública del matcher (Rust) 19
6.1. Crate de entrada y módulos . . . 19
6.2. Tipos públicos . . . 19
6.3. Construcción del grafo . . . 20
6.4. Matching: API principal . . . 20
6.4.1. API LFTJ (worst-case-optimal) . . . 21
6.5. Introspección de runtime . . . 21
6.6. Compatibilidad con la API previa . . . 21
6.7. Concurrencia . . . 21
6.8. Errores . . . 22
6.9. Versionado . . . 22

7. Anomaly scoring 23
7.1. Composición . . . 23
7.2. Componente 1: Entity Rarity . . . 23
7.3. Componente 2: Edge Rarity . . . 23
7.4. Componente 3: Neighborhood Concentration . . . 24
7.5. Componente 4: Temporal Novelty . . . 24
7.6. Componente 5: GNN Threat . . . 24
7.7. Explicación . . . 24
7.8. Invariantes . . . 25

8. Variables de entorno **26**
8.1. Convención . . . 26
8.2. Daemon (Tauri app) . . . 27
8.3. Cliente MCP . . . 27
8.4. Integraciones externas . . . 27
8.5. Feature flags . . . 28
8.6. Flags de *performance* . . . 28
8.7. Fuente canónica . . . 30

9. Errores y códigos **31**
9.1. HTTP API . . . 31
9.2. MCP tools . . . 31
9.3. DSL parser . . . 32
9.4. C ABI (histórica) . . . 32
9.5. Ingesta . . . 32
9.6. Convención de log . . . 33

A. Historial de cambios de superficie **34**
A.1. Política de versiones . . . 34
A.2. v1.0 — baseline estable . . . 34
A.3. Compromisos v1 . . . 35
A.4. Qué queda fuera de v1 . . . 35
A.5. Tags git de release . . . 35
A.6. Fuente canónica . . . 35

Capítulo 1

Introducción

Este libro es la *especificación* de las superficies externas de GraphHunter. Enumera, con contrato, todo lo que el sistema expone al mundo: los 64 endpoints HTTP, las 55 tools MCP, la gramática del DSL de hipótesis, la C ABI histórica de `libgraphmatch` (motor eliminado el 2026-05-07; capítulo conservado como referencia), los formatos de log aceptados, las variables de entorno y los códigos de error.

Véase también el libro *Algoritmos* (`docs/algoritmos/algoritmos.pdf`) para entender el *por-qué* detrás de cada *contract* — por ejemplo, por qué hay tantas variables de entorno relacionadas a *indexes* (heavy-logs #1–#4) o cómo el separador `;` de la DSL se conecta con el *tree-walk Yannakakis enumerator*.

1.1. Para qué sirve esta especificación

Un cliente del sistema — humano o automatizado — necesita saber qué puede pedir, qué va a recibir y qué cambios de comportamiento pueden esperarse entre versiones. Este libro responde esas tres preguntas en forma de contratos. Cada contrato indica:

- Su **estabilidad**: `stable`, `experimental` o `deprecated`.
- Su **versión** (semántica) cuando aplica.
- Los **tipos de entrada y salida**, con un ejemplo.
- Los **códigos de error** que puede emitir.

1.2. Qué no está aquí

Los algoritmos, los patrones de implementación y las decisiones arquitectónicas viven en el libro de Arquitectura. El flujo de uso para un analista vive en el Manual. La justificación experimental de la composición de scoring y la equivalencia DFS/SIMD vive en el Paper.

1.3. Estabilidad y versionado

El sistema adhiere a semver con un matiz: cada superficie tiene su propia curva. Los DTOs de la HTTP API viven en `platform/api/src/dto/v1/` y están versionados como `v1`. Las tools MCP llevan `version` por tool (ver ADR-004 en el libro de Arquitectura). El DSL tiene `v1` estable y `v2-experimental` feature-gated. La C ABI lleva `gm_abi_version()`.

La regla operativa: **no** se rompe silenciosamente una superficie `stable`. Si una release cambia el comportamiento visible, sube `version` y se documenta el cambio en el apéndice A.

1.4. Convenciones del libro

- **Endpoints HTTP**: se listan como `METHOD /ruta` con descripción corta, payload JSON de request y response, códigos de error relevantes.
- **Tools MCP**: se listan con `nombre`, `domain`, `stability`, `version`, input schema y output schema.
- **Funciones C**: se listan con signature, pre/post condiciones y ownership semántico.
- **Producciones BNF** para la gramática DSL.
- **Env vars**: nombre, tipo esperado, default, componente que la consume.

1.5. Mapa del libro

- Cap. 2 — HTTP API: 64 endpoints.
- Cap. 3 — MCP: 55 tools en 10 dominios.
- Cap. 4 — DSL de hipótesis: gramática BNF y ejemplos.
- Cap. 5 — Formatos de log aceptados por los parsers.
- Cap. 6 — C ABI histórica de `libgraphmatch` (referencia, motor removido).
- Cap. 7 — Scoring: cinco componentes con dominio y fórmula.
- Cap. 8 — Variables de entorno.
- Cap. 9 — Errores por superficie.
- Apéndice A — Changelog de superficies.

Capítulo 2

HTTP API canónica

2.1. Transporte y autenticación

El Tauri daemon sirve la API en `http://127.0.0.1:\$GRAPHHUNTER_API_PORT` (default `37891`, configurable por env var `GRAPHHUNTER_API_PORT`). Por defecto sólo se bindea a loopback; el host se puede cambiar con `GRAPHHUNTER_API_HOST`.

Autenticación: cada request debe presentar el header `Authorization: Bearer \$GRAPHHUNTER_API_TOKEN`. El token se genera al instalar la app y se rotacional con el comando `cli auth rotate`. Requests sin token reciben 401.

Formato: request y response en `application/json` UTF-8. Timestamps ISO-8601 con offset explícito. Todos los identificadores internos de entidad son enteros sin signo de 64 bits.

2.2. Esquema introspectivo

- `GET /v1/schema` — Devuelve el listado completo de endpoints, sus DTOs y su estabilidad. Base para clientes generados automáticamente. Incluye además `schema_compute_time_ms` y `cache_hit` para diagnóstico.

2.3. Salud y telemetría

- `GET /` — Root; info de build y versión.
- `GET /health` — Liveness / readiness combinado.
- `GET /metrics` — Contadores internos (hunts ejecutados, eventos ingeridos, errores por categoría). Formato compatible con Prometheus scrape.

2.4. Catálogo y schema

- `GET /catalog` — Catálogo de tipos de entidad y relaciones del workspace.
- `GET /catalog/status` — Catálogo con flags de salud por entrada.
- `GET /catalog/diagnostics` — Resultados de verificación estructural.
- `GET /entity_types` — Lista de tipos de entidad con conteos.
- `GET /entity_type_counts` — Histograma de conteos.
- `GET /relation_schema` — Matriz `(from, relation, to)` observada en el grafo.

2.5. Navegación del grafo

- `GET /search` — Búsqueda de entidades por valor o tipo. Parámetros: `q`, `type`, `limit`.
- `GET /expand` — Vecinos de un nodo.
- `GET /expand_grouped` — Vecinos agrupados por relation type (para UIs densas).
- `GET /node_details` — Propiedades y eventos asociados a un nodo.
- `POST /subgraph` — Subgrafo inducido por un conjunto de nodos.
- `GET /events_for_node` — Eventos donde participa un nodo.
- `GET /events_paginated` — Eventos con paginación (cursor-based, evita OOM sobre grafos grandes).
- `GET /path_nodes` — Nodos en un path concreto devuelto por un hunt.
- `GET /graph_summary` — Conteos agregados del grafo.

2.6. Caza (hunts)

- `POST /run_hunt` — Ejecuta una hipótesis contra el grafo actual. Body: `\{dsl: string, min_score?: f32, max_results?: u32\}`. Devuelve `HuntResult\{paths, scores, explanation\}`.
- `GET /hunt_results` — Últimos resultados persistidos (paginados).
- `GET /explain_score` — Desglose de los cinco componentes de anomaly scoring para un path.
- `POST /diff_hunts` — Diferencia entre dos hunts por ID.
- `POST /parse_dsl` — Valida y devuelve AST de una hipótesis DSL sin ejecutar.
- `POST /compute_scores` — Recomputa scoring sobre resultados existentes (tras cambiar componentes).
- `POST /enable_anomaly_scoring` — Toggle de scoring anómalo a nivel sesión.

2.7. Ingesta

- `POST /ingest_negotiate` — Acuerda con el cliente formato y mapping antes de abrir el stream.
- `POST /canonical_map` — Aplica un `FieldConfig` para normalizar un lote de eventos.
- `POST /parser_generate` — Deriva un `FieldConfig` heurístico a partir de un preview.
- `POST /schema_drift_detect` — Compara un batch contra el esquema conocido y reporta drift.
- `POST /mapping_regression_test` — Corre un mapping sobre un set congelado para detectar regresiones.
- `POST /publish_mapping` — Publica un mapping al `mapping_library`.
- `POST /fetch_shared_mappings` — Obtiene mappings compartidos del library.

2.8. Anotación

- `POST /notes`, `POST /notes/` — Crea una nota sobre una entidad, path o hunt.
- `GET /notes/list` — Lista paginada de notas.
- `GET /notes/:id` — Nota puntual.
- `POST /tag` — Etiqueta una entidad.
- `POST /untag` — Remueve una etiqueta.
- `GET /tags` — Lista de entidades por etiqueta.

2.9. Anomaly y enrichment

- `GET /temporal_heatmap` — Matriz hora × día de actividad.
- `GET /subnet_analysis` — Análisis por subnet (densidad, entidades top, eventos anómalos).
- `POST /enrich_ip` — Enriquecimiento online de una IP (GeoIP, WHOIS cached).

2.10. Exportación

- `POST /export_iocs` — Exporta IOCs (STIX-like).
- `POST /publish_iocs_to_sentinel` — Publica IOCs a Azure Sentinel. Requiere `GRAPHHUNTER_IOC_PUBLI`
- `POST /save_hypothesis_as_sigma` — Convierte una hipótesis DSL a regla Sigma.
- `POST /file_ticket` — Abre ticket en el sistema configurado. Requiere `GRAPHHUNTER_TICKET_PUBLISH_EN`
- `POST /export_subgraph` — Exporta subgrafo en formato Graphson / JSONLines.
- `POST /export_ocsf` — Exporta eventos en OCSF v1.4.
- `POST /export_hunt_results` — Exporta resultados del último hunt en CSV/JSON.

2.11. Invariantes

- `POST /check_invariants` — Corre checks estructurales sobre el grafo (treewidth, shape catalog, integridad temporal).
- `POST /invariant_check_hypothetical` — Corre los mismos checks como *dry-run* sobre un grafo candidato (ingestión previa a commit).

2.12. Agentic (slow lane)

- `POST /agentic_review_list` — Lista propuestas pendientes de review.
- `POST /agentic_review_approve` — Aprueba una propuesta.
- `POST /agentic_review_reject` — Rechaza una propuesta con razón.

2.13. Dead Letter Queue

- `POST /list_dead_letters` — Lista eventos que fallaron ingesta.
- `POST /reingest_dead_letter` — Reintenta un evento (tras corregir mapping o parser).
- `POST /purge_dead_letters` — Borra entradas de la DLQ.

2.14. SIEM (Sentinel)

- `POST /kql` — Ejecuta una query KQL contra Sentinel configurado y persiste resultados al workspace.
- `GET /sentinel_status` — Estado del conector y de la última sesión.

2.15. Datasets y utilidad

- `GET /datasets` — Datasets disponibles en `demo_data/`.

2.16. Códigos de respuesta

- `200` — éxito (con body JSON).
- `204` — éxito sin body (operaciones idempotentes).
- `400` — request inválido (DTO mal formado).
- `401` — token ausente o inválido.
- `403` — feature flag desactivado (p.ej. IOC publish sin env var).
- `404` — recurso inexistente (nodo, hunt, nota).
- `409` — conflicto (ingest en curso, session locked).
- `422` — validación semántica falló (DSL inválido, schema drift fuera de tolerancia).
- `500` — error interno; revisar logs del daemon.

2.17. Total

El daemon expone 64 rutas distintas (algunas con y sin slash final). De ellas, 62 son endpoints de producto; `/` y `/metrics` son meta. El listado `canónico` del runtime (`GET /v1/schema`) es la fuente de verdad si hay discrepancia con este documento.

Capítulo 3

Herramientas MCP

3.1. Protocolo

El servidor MCP (Model Context Protocol) expone **55 tools** organizadas en 10 dominios. Cada tool se define con la interfaz `Tool<I, O>` en `platform/mcp/src/lib/types.ts`:

```
export interface Tool<I = unknown, O = unknown> {
  readonly name: string;
  readonly domain: string;
  readonly description: string;
  readonly stability: 'stable' | 'experimental' | 'deprecated';
  readonly version: string;
  readonly inputSchema: z.ZodType<I>;
  readonly outputSchema?: z.ZodType<O>;
  run(input: I, ctx: ToolContext): Promise<O>;
}
```

Cada tool es un archivo en `platform/mcp/src/tools/<dominio>/<nombre>.ts` y se registra vía el `index.ts` del dominio. Los 10 dominios están listados en las secciones siguientes.

3.2. Dominio `agentic` (9 tools)

Flujo agentic slow-lane: negociación de ingesta, generación de parser asistida, revisión humana, publicación de mappings compartidos.

- `agentic_review_list` — lista propuestas pendientes.
- `agentic_review_approve` — aprueba.
- `agentic_review_reject` — rechaza.
- `canonical_map` — normaliza eventos vía `FieldConfig`.
- `fetch_shared_mappings` — trae mappings del library.
- `field_config_schema` — schema actual de `FieldConfig`.
- `ingest_negotiate` — handshake de ingesta.
- `parser_generate` — infiere `FieldConfig`.
- `publish_mapping` — publica un mapping validado.

3.3. Dominio `catalogs` (4 tools)

- `get_catalog` — catálogo de entity/relation types.
- `get_catalog_diagnostics` — checks de integridad.
- `get_catalog_with_status` — catálogo con banderas de salud.
- `list_datasets` — datasets disponibles.

3.4. Dominio `data-quality` (7 tools)

- `invariant_checker` — corre invariantes del grafo.
- `invariant_check_hypothetical` — dry-run antes de ingestar.
- `mapping_regression_test` — prueba mapping contra fixture congelado.
- `schema_drift_detect` — compara batch contra esquema conocido.
- `list_dead_letters` — lista DLQ.
- `reingest_dead_letter` — reintenta evento.
- `purge_dead_letters` — borra entradas DLQ.

3.5. Dominio `export` (3 tools)

- `export_iocs` — IOCs STIX-like.
- `export_ocsf` — OCSF v1.4.
- `export_subgraph` — subgrafo como GraphSON/JSONL.

3.6. Dominio `graph-views` (4 tools)

- `get_entity_type_counts` — histograma de tipos.
- `get_graph_summary` — conteos globales.
- `get_subnet_analysis` — vista por subnet.
- `get_temporal_heatmap` — matriz hora $\times$ día.

3.7. Dominio `hunting` (6 tools)

- `run_hunt` — ejecuta hipótesis DSL.
- `get_hunt_results` — resultados persistidos.
- `diff_hunts` — diferencia entre dos hunts.
- `export_hunt_results` — CSV/JSON.
- `parse_dsl` — valida DSL sin ejecutar.
- `save_hypothesis_as_sigma` — genera regla Sigma.

3.8. Dominio integrations (3 tools)

- enrich_ip — GeoIP/WHOIS de una IP.
- file_ticket — abre ticket en el sistema externo.
- publish_iocs_to_sentinel — empuja IOCs a Sentinel.

3.9. Dominio navigation (10 tools)

- search_entities — búsqueda por valor/tipo.
- expand_node — vecinos de un nodo.
- expand_node_grouped — vecinos agrupados por relación.
- check_connection — ping + auth check.
- get_entity_types — lista de tipos.
- get_events_for_node — eventos del nodo.
- get_events_paginated — eventos paginados.
- get_node_details — propiedades + eventos.
- get_relation_schema — matriz (from, rel, to).
- get_subgraph — subgrafo inducido.

3.10. Dominio notes (6 tools)

- create_note — crea nota sobre entidad/path/hunt.
- get_notes — lista.
- tag_entity — etiqueta.
- untag_entity — remueve etiqueta.
- list_tagged — entidades por etiqueta.
- get_path_nodes — nodos de un path concreto.

3.11. Dominio scoring (3 tools)

- enable_anomaly_scoring — toggle por sesión.
- compute_scores — recomputa scoring sobre resultados existentes.
- explain_score — desglose de los 5 componentes para un path.

3.12. Contrato común

Todas las tools MCP comparten:

- Autenticación: la API key del MCP se deriva de `GRAPHHUNTER_API_TOKEN`; el server MCP actúa como proxy tipado del API HTTP.
- Errores: cualquier error del API subyacente se propaga como `\{error: code, message, details\}` estructurado.
- Timeouts: 30s default para tools de lectura, 300s para tools de ingesta / hunt.
- Idempotencia: las tools marcadas como `stable` son seguras de reintentar con el mismo input.

3.13. Tests contractuales

El directorio `platform/mcp/test/` contiene tests que validan cada tool contra un `platform/api` mock según el schema del endpoint correspondiente. Un mismatch entre la definición de la tool y el DTO del API rompe el build antes de release.

Capítulo 4

Lenguaje de hipótesis (DSL)

4.1. Propósito

El DSL de GraphHunter describe *hipótesis* de caza: patrones que buscamos en el grafo. Una hipótesis es una secuencia de *steps*: entidades conectadas por relaciones, potencialmente con restricciones temporales y predicados.

4.2. Gramática v1 (estable)

En notación BNF extendida:

```
hypothesis  ::= step (relation step)* constraints?

step        ::= entityType ( '{' predicates '}' )?

entityType  ::= 'IP' | 'Host' | 'User' | 'Process'
              | 'File' | 'Domain' | 'Registry' | 'URL'
              | 'Service' | '*'
              | customEntityType           (*@\textrm{si hay extensiones}@*)

relation    ::= '-[' relationType ']->'
              | '<-[' relationType ']-'

relationType::= 'Auth' | 'Connect' | 'Execute' | 'Read'
              | 'Write' | 'DNS' | 'Modify' | 'Spawn'
              | 'Delete' | '*'
              | customRelationType         (*@\textrm{si hay extensiones}@*)

predicates  ::= predicate (',' predicate)*
predicate   ::= field op value
              | builtinPredicate '(' args ')'

constraints ::= '{' constraint (',' constraint)* '}'
constraint  ::= 'k'      '=' int         (*@\textrm{k-simplicity cap}@*)
              | 'window' '=' duration    (*@\textrm{ventana temporal}@*)
              | 'order'  '=' 'temporal' | 'any'

duration    ::= int ('s'|'m'|'h'|'d')
```

4.3. Entity types built-in

Tipo	Semántica
`IP`	dirección IPv4/IPv6
`Host`	nombre de host o FQDN
`User`	principal de identidad (usuario local, AAD, etc.)
`Process`	proceso (nombre de ejecutable + hash opcional)
`File`	archivo en filesystem
`Domain`	nombre de dominio DNS
`Registry`	clave/valor de registro Windows
`URL`	URL
`Service`	servicio / aplicación SaaS
`*`	comodín: cualquier tipo

Tabla 4.1: Tipos de entidad incluidos en la gramática v1.

4.4. Relation types built-in

Relación	Uso típico
`Auth`	User → Host / Service: autenticación
`Connect`	IP → IP / Host: conexión de red
`Execute`	Process → File / Process: invocación
`Read`	Process → File / Registry: lectura
`Write`	Process → File / Registry: escritura
`DNS`	Host → Domain: resolución
`Modify`	Process → cualquiera: modificación
`Spawn`	Process → Process: padre-hijo
`Delete`	Process → File: borrado
`*`	comodín

Tabla 4.2: Tipos de relación en la gramática v1.

4.5. Predicados built-in

- `in_subnet(ip, cidr)` — IP en un bloque CIDR.
- `matches_regex(field, pattern)` — campo contra regex PCRE-like.
- `is_weekend()` — evento ocurrió fuera de horario laboral.
- `within_hours(start, end)` — ventana horaria.
- `tag(name)` — entidad tiene la etiqueta indicada.

Comparadores escalares disponibles: `=`, `!=`, `>`, `>=`, `<`, `<=`. Strings soportan `like`.

4.6. Ejemplos

Lateral movement simple.

```
User -[Auth]-> Host -[Connect]-> Host {k=3}
```

Busca un usuario que autenticó contra un host, y desde ese host se abrió una conexión a otro host. La cota `k=3` limita la explosión combinatoria (simplicidad de hipergrafo).

Proceso que escribe en directorio sensible.

```
Process -[Write]-> File {path like "C:\\Windows\\System32\\*"}
```

DNS tunneling sospechoso.

```
Host -[DNS]-> Domain {matches_regex(value, "^[a-f0-9]{32}\\.")}
{window=1h}
```

Hosts que resuelven muchos dominios con prefijo hex de 32 caracteres dentro de una ventana de 1h.

Cadena de spawning.

```
Process {name="winword.exe"}
  -[Spawn]-> Process {name="cmd.exe"}
  -[Spawn]-> Process {name="powershell.exe"}
```

4.7. Extensibilidad (ADR-005)

El trait `DslExtension` permite registrar:

- **Entity types custom**: `Container`, `ServiceAccount`, `Cluster`, etc.
- **Relation types custom**.
- **Predicados custom**: implementan `BuiltinPredicate` trait.

Una extensión registra su símbolo y su implementación en el `ExtensionRegistry` al arrancar. La gramática v2 (detrás de `feature = "dsl-v2-experimental"`) incluye operadores temporales explícitos (`before`, `within`) y notación para rangos.

4.8. Compatibilidad

Garantía v1: una hipótesis escrita hoy contra v1 sigue parseando y comportándose igual en todas las versiones futuras que mantengan `grammar v1`. Los cambios compatibles agregan símbolos o predicados; los cambios incompatibles promueven a v2.

Capítulo 5

Formatos de log soportados

5.1. El trait `LogParser`

El punto de extensión está en `platform/parsers/src/lib.rs`:

```
pub trait LogParser: Send + Sync {
    fn name(&self) -> &str;
    fn can_parse(&self, sample: &[u8]) -> bool;
    fn parse(&self, bytes: &[u8]) -> Result<Vec<Event>>;
    fn field_roles(&self) -> &FieldConfig;
}
```

El dispatcher `format_registry` en `platform/api/src/format_registry.rs` consulta `can_parse` sobre una muestra del archivo y elige el primer parser que responde positivamente. El fallback es el parser `generic` con heurísticas.

Cada parser produce un `Vec<Event>`; cada `Event` es una tupla `(timestamp, actor, action, target, attrs)` con roles de campo ya asignados.

5.2. Sysmon (Windows Event Log XML)

Fuente: archivos EVTX exportados, o XML de `Microsoft-Windows-Sysmon/Operational`. Parser: `platform/parsers/src/sysmon.rs` + lógica antigua en `core/graph-engine/src/sysmon.rs`.

Event IDs cubiertos: 1 (process create), 3 (network connect), 7 (image load), 8 (CreateRemoteThread), 10 (ProcessAccess), 11 (FileCreate), 13 (RegistryValueSet), 22 (DNS), 23 (FileDelete).

Mapeo a entidades:

- `Image`, `ParentImage` → `Process`.
- `User` → `User`.
- `Computer` → `Host`.
- `DestinationIp` → `IP`.
- `QueryName` → `Domain`.
- `TargetFilename` → `File`.

Relaciones típicas: `Process -[Spawn]-> Process` (ID 1), `Host -[Connect]-> IP` (ID 3), `Process -[Write]-> File` (ID 11).

5.3. Sentinel / Log Analytics (JSON)

Fuente: resultados de query KQL exportados como JSON o streamed via conector. Parser: `platform/parsers/src/sentinel.rs`.

Tablas soportadas: `SigninLogs`, `AuditLogs`, `SecurityEvent`, `DeviceProcessEvents` (Defender for Endpoint), `AzureActivity`, `SecurityAlert`.

Mapeo: depende de la tabla; el parser resuelve vía `FieldConfig` propio de cada una. Ejemplo para `SigninLogs`: `UserPrincipalName` → `User`, `IPAddress` → `IP`, `AppDisplayName` → `Service`, con relación `User -[Auth]-> Service`.

5.4. CSV genérico

Parser: `platform/parsers/src/generic_csv.rs`. Requiere un `FieldConfig` explícito (producido por `parser_generate` o aportado por el usuario vía la UI).

Roles de campo: `TIME`, `ACTOR`, `ACTION`, `TARGET`, `INFO`. El rol `TIME` es obligatorio; el parser falla si no logra asignar uno.

5.5. Generic (heurístico)

Parser: `core/graph-engine/src/generic.rs`. Dado un archivo de texto con líneas KV o JSON-lines, intenta:

- Detectar timestamps con un catálogo de formatos comunes.
- Reconocer direcciones IP, dominios, hashes.
- Inferir relaciones por colocación de campos.

Es el fallback. Siempre produce algún grafo, pero las relaciones pueden ser de baja calidad. La UI marca este parser con `stability = experimental` en la sesión resultante.

5.6. FortiAnalyzer

Parser: `platform/parsers/src/forti.rs`. CSV propietario con campos del estilo `date`, `time`, `srcip`, `dstip`, `dstport`, `app`, `action`, `user`.

Mapeo:

- `srcip`, `dstip` → `IP`.
- `user` → `User`.
- `app` → `Service`.

Relaciones: `IP -[Connect]-> IP` con atributos `dstport`, `app`, `action`.

5.7. AWS Cognito

Parser: `platform/parsers/src/cognito.rs`. JSON de CloudWatch Logs para eventos de Cognito (signin, signup, token refresh).

Mapeo: `username` → `User`, `sourceIp` → `IP`, `userPoolId` → `Service`.

5.8. IIS (W3C Extended Log)

Parser: `platform/parsers/src/iis.rs`. Formato W3C con `\#Fields` header. Maneja aliases comunes (`cs-username`, `c-ip`, `cs-uri-stem`, etc.).

Mapeo: `cs-username` → `User`, `c-ip` → `IP`, `cs-uri-stem` → `URL`.

5.9. Tres carriles de ingesta

La ingesta productiva corre sobre el *fast lane*: un programa VRL compilado determinísticamente desde un `FieldConfig` aprobado. Los parsers listados arriba son fuentes de `FieldConfig`; el código que ejecuta por evento en runtime es VRL, no el parser. Los parsers siguen activos para ingesta legacy pre-VRL y como oráculo en `mapping_regression_test`.

El detalle arquitectónico vive en el libro de Arquitectura, capítulo *Ingestor agéntico*. Esta sección cubre lo que un cliente del sistema necesita saber para escribir contra la superficie.

5.9.1. Fast lane — VRL compilado

Dado un `FieldConfig` f, el compilador emite un programa VRL textual con identidad estable:

$$\text{VRL}(f) = \text{compile}(f) \qquad \text{mapping_hash}(f) = \text{sha256}(\text{VRL}(f))$$

El `mapping_hash` se estampa en el campo `provenance.mapping_hash` de cada evento canonical emitido. Garantiza que dos eventos con el mismo hash fueron producidos por la misma lógica de ingesta.

Contrato del compilador (`platform/vrl`):

- Entrada: `FieldConfig` válido (semántica del capítulo anterior).
- Salida: `VrlProgram` (`struct` con el source VRL, el hash, y metadatos de la versión del compilador).
- Pura: sin I/O, sin red, sin LLM. La misma entrada produce idéntico output byte-a-byte.
- El runtime VRL ejecuta el programa sobre cada evento crudo y emite canonical OCSF.

5.9.2. Slow lane — LLM-as-compiler

La tool MCP `parser_generate` (y su endpoint HTTP `POST /parser_generate`) toma una muestra del formato y devuelve un `FieldConfig` propuesto. La implementación usa un backend LLM (`platform/local-llm`) que puede ser local (GGUF) o remoto (API). El output jamás se aplica automáticamente: entra a la cola de review agéntica vía `POST /agentic_review_list`.

Flujo típico:

1. Cliente llama `parser_generate` con `\{sample: "...", format_hint: "fortios"\}`.
2. Server devuelve `\{proposal_id, field_config, ocsf_mapping, confidence\}`.
3. Humano abre `agentic_review_list`, aprueba o edita vía `agentic_review_approve` o `agentic_review_reject`.
4. Tras aprobación, `publish_mapping` persiste el mapping en `platform/mapping-library` con `mapping_hash` estable.
5. Próxima ingesta con ese formato usa automáticamente el nuevo mapping.

5.9.3. Learning lane — drift-as-signal

Cada batch en fast lane produce métricas estructurales:

- Histograma de entity types emitidos.
- Cardinalidad de cada campo canonical.
- Ratio de eventos enviados a DLQ.
- Distribución de timestamps observados.

El endpoint `POST /schema_drift_detect` (tool MCP `schema_drift_detect`) toma un batch y devuelve un score de drift por campo usando Kolmogorov-Smirnov contra el perfil del mapping actual. Si el drift supera tolerancia, el sistema:

1. **No** para la ingesta — sigue usando el mapping actual.
2. Abre propuesta de re-review en la cola agéntica.
3. El slow lane procesa la propuesta; humano decide si aceptar la evolución.

El contrato es explícito: drift no es error, es señal. El sistema nunca deja de ingestar por descubrir un cambio aguas arriba.

5.10. Canonical OCSF

Todo mapping emite, como salida canonical, eventos con shape **OCSF v1.4** (Open Cybersecurity Schema Framework). Campos principales garantizados:

Campo	Semántica
`time`	timestamp ISO-8601 UTC
`category_uid`	categoría OCSF (ej. 4 Network)
`class_uid`	clase específica
`activity_id`	acción normalizada
`src_endpoint.ip`	origen IP
`dst_endpoint.ip`	destino IP
`actor.user.name`	principal
`device.hostname`	host
`observables[]`	entidades extraídas para el grafo
`provenance.mapping_hash`	hash del VRL que produjo este evento
`provenance.source_format`	formato raw de origen

Tabla 5.1: Subconjunto de campos OCSF garantizados.

Cualquier campo adicional específico del formato raw se emite bajo `raw.\{...\}` como escape hatch: el grafo no lo usa pero el analista puede consultarlo.

5.11. FieldConfig como contrato

Todos los parsers convergen a un `FieldConfig` que describe roles y mapeos. Este FieldConfig se puede:

- **Inferir** vía `parser_generate` (el sistema propone, el usuario confirma).
- **Compartir** vía `publish_mapping` al `mapping_library`.

- **Versionar**: cada mapping tiene su versión propia; cambios requieren nueva versión.
- **Compilar a VRL** vía `platform/vrl` para portarlo a pipelines externos (Vector, Fluent Bit).

5.12. Cómo agregar un parser

Resumen (el detalle vive en el libro de Arquitectura, capítulo *Capa Platform*):

1. Crear el archivo en `platform/parsers/src/<nombre>.rs`.
2. Implementar `LogParser`.
3. Registrar en `platform/parsers/src/lib.rs`.
4. Agregar fixtures a `platform/parsers/tests/`.
5. Agregar regla de detección a `format_registry`.

No es necesario tocar ninguna otra capa.

Capítulo 6

Superficie pública del matcher (Rust)

Estado (2026-05-12). Hasta 2026-05-07 esta especificación describía una C ABI estable (`gm\_graph\_t`, `gm\_matcher\_t`, `gm\_results\_t`) expuesta por `libgraphmatch` y consumida por `core/matcher-ffi` vía FFI. La C ABI se eliminó junto con el motor C++. La superficie pública actual es Rust pura y vive en `core/graph-engine`. El contrato histórico de la C ABI se conserva en `docs/\_archive/architecture-es/chapters/ch11-simd.tex`. La etiqueta `sec:cabi` se preserva como ancla legacy para los cross-refs (`\textbackslash ref\{sec:cabi\}`) que ya apuntan a este capítulo.

6.1. Crate de entrada y módulos

- `core/graph-engine` (`graph\_hunter\_core`): motor de matching, intersección SIMD, anomaly scoring, persistencia, LFTJ. Es el árbol que reemplazó a `libgraphmatch`.

- `core/matcher-ffi` (`graph\_hunter\_matcher\_ffi`): *thin forwarder* Rust puro ≈30 LOC sobre `graph\_hunter\_core`. No tiene `build.rs`, no depende de `cc-rs` ni `bindgen`, no contiene `extern "C"`. Candidato a eliminación una vez que `platform/api` llame al engine directamente.

La distribución es un *único binario Rust* por target. Ya no hay `.so`/`.dll`/`.a` nativos que verificar al arranque.

6.2. Tipos públicos

- `GraphHunter` (`core/graph-engine/src/graph.rs`): el grafo y su matcher. Owner de los buffers de aristas, `StringInterner`, `entity\_type\_tags`, opcionalmente un `AnomalyScorer` finalizado.

- `Hypothesis` (`core/graph-engine/src/hypothesis.rs`): descripción del patrón a buscar (tipo de start node, secuencia de pasos, ventana temporal, predicados de metadata).

- `HuntResult` (`core/graph-engine/src/graph.rs`): un match. Contiene `path: Vec<Arc<str>>`, `timestamps: Vec<i64>`, `score: f32`. La migración a `Arc<str>` (2026-04-30) eliminó el 55 % de costo cap-saturado original.

- `GraphError` (`core/graph-engine/src/errors.rs`): errores enumerados (`InvalidHypothesis(\_)`, `TimeRange`, `Capacity`, `Internal(\_)`, etc.). Toda función pública devuelve `Result<\_, GraphError>`.

6.3. Construcción del grafo

```
impl GraphHunter {
    pub fn new() -> Self;
    pub fn with_capacity(entities: usize, edges: usize) -> Self;

    pub fn add_entity(
        &mut self,
        symbol: &str,
        entity_type: EntityType,
    ) -> Result<StrId, GraphError>;

    pub fn add_relation(
        &mut self,
        src: &str,
        dst: &str,
        rel: RelationType,
        timestamp_ms: i64,
    ) -> Result<(), GraphError>;

    pub fn finalize(&mut self) -> Result<(), GraphError>;
}
```

Ownership: el caller posee el `GraphHunter`. La desalocación es automática vía `Drop` del lado Rust; no hay funciones explícitas `*\_free`. La validación de duplicados y rangos vive en el cuerpo de `add\_entity`/`add\_relation`, no en `finalize`.

6.4. Matching: API principal

```
impl GraphHunter {
    /// DFS iterativo cap-acotado. `max_results` default = 10_000.
    /// Devuelve `(matches, has_more)` - `has_more` indica saturacion.
    pub fn search_temporal_pattern(
        &self,
        hypothesis: &Hypothesis,
        time_window: Option<(i64, i64)>,
        max_results: Option<usize>,
    ) -> Result<(Vec<HuntResult>, bool), GraphError>;

    /// DFS smart: top-k con poda por anomaly score. Requiere un
    /// `AnomalyScorer` finalizado en `self.anomaly_scorer`.
    pub fn search_temporal_pattern_smart(
        &self,
        hypothesis: &Hypothesis,
        time_window: Option<(i64, i64)>,
        top_k: usize,
    ) -> Result<(Vec<HuntResult>, bool), GraphError>;
}
```

Precondiciones: el grafo debe estar finalizado (`finalize()` fue llamada). Una llamada sobre un grafo no finalizado devuelve `Err(GraphError::Internal(\_))` en debug y un resultado vacío en release; el contrato es "no hacerlo", no "recuperarse de hacerlo".

Thread-safety: tras `finalize`, el `GraphHunter` es `Send + Sync`: múltiples hilos pueden correr `search\_*` simultáneamente sobre el mismo grafo. Antes de `finalize`, el contrato es "construir desde un solo hilo".

6.4.1. API LFTJ (worst-case-optimal)

Para patrones cíclicos pequeños existe una familia de entrypoints LFTJ que reutilizan un `MaterializedCsrTrie` construido a demanda:

- `search\_temporal\_pattern\_triangle(\_with\_trie)` — K_3.
- `search\_temporal\_pattern\_k4\_lftj(\_with\_trie)` — K_4.
- `search\_temporal\_pattern\_c6\_lftj(\_with\_trie)` — ciclo C_6 usando forward + reverse trie.
- `search\_temporal\_pattern\_k5\_lftj(\_with\_trie)` — K_5.

Las variantes `\_with\_trie` permiten amortizar el trie sobre múltiples queries. Todas comparten la firma `Result<(Vec<HuntResult>, bool), GraphError>`.

6.5. Introspección de runtime

```
// core/graph-engine/src/simd_rust.rs
pub fn isa_label() -> &'static str;  // "avx2" | "scalar"
```

`isa\_label()` hace runtime detection vía `is\_x86\_feature\_detected!("avx2")` cada vez que se llama; no hay caching. La superficie de la API la expone como string en `platform/api::operations::gra`

```
pub fn runtime_info() -> RuntimeInfo {
    RuntimeInfo {
        simd_isa: graph_hunter_core::simd_rust::isa_label().to_string(),
    }
}
```

Y la UI Tauri llega via el command `cmd\_get\_simd\_isa` (`apps/tauri/src-tauri/src/commands/graph\_ops`

6.6. Compatibilidad con la API previa

Para evitar tocar call-sites en `platform/api` en el mismo PR de la migración, `core/matcher-ffi` retiene la firma libre:

```
// core/matcher-ffi/src/lib.rs
pub fn search_temporal_pattern_simd(
    hunter: &GraphHunter,
    hypothesis: &Hypothesis,
    time_window: Option<(i64, i64)>,
    max_results: Option<usize>,
) -> Result<(Vec<HuntResult>, bool), GraphError>;
```

Despacha a `\_smart` si hay scorer finalizado, sino a `search\_temporal\_pattern`. El nombre con sufijo `\_simd` es histórico — ya no implica un codepath separado de C++.

6.7. Concurrencia

- **Tras finalize**: el `GraphHunter` cumple `Send + Sync`. Hunts paralelos sobre el mismo grafo son válidos por contrato (rayon-friendly).
- **Pre finalize**: construir desde un solo hilo. El `StringInterner` interno no es thread-safe en modo insertion.

- **Anomaly scorer**: si está, se asume finalizado al arrancar el matcher; mutarlo concurrentemente con un hunt es UB lógico (no UB de memoria gracias al borrow checker, pero la salida es indeterminada).

6.8. Errores

A diferencia de la C ABI, la API Rust no usa convenciones "`NULL` significa error". Toda función fallible devuelve `Result<T, GraphError>` y el caller patternmatches sobre el discriminante. Ver cap. 9 para el listado completo.

6.9. Versionado

No hay ABI version check al arranque. El crate Rust se versiona por semver (`graph\_hunter\_core = "0.x"`) y la compatibilidad es source-level, no binary-level. La idea "ABI mismatch = abort temprano" (ADR-002 en el libro *Arquitectura*, apéndice de ADRs) perdió relevancia porque ya no hay biblioteca dinámica que pueda desincronizarse.

Capítulo 7

Anomaly scoring

7.1. Composición

GraphHunter asigna a cada path resultado de un hunt un *score* compuesto en $[0, 1]$, calculado como combinación ponderada de cinco componentes independientes.

$$S(p) = \sum_{i=1}^{5} w_i \cdot C_i(p) \qquad \sum_i w_i = 1$$

Los pesos w_i están en `ScoringWeights` (ver `core/graph-engine/src/anomaly.rs`). Valores por defecto:

Campo	Peso default	Componente
`w1_entity_rarity`	0.25	Entity Rarity
`w2_edge_rarity`	0.30	Edge Rarity
`w3_neighborhood_conc`	0.25	Neighborhood Concentration
`w4_temporal_novelty`	0.20	Temporal Novelty
`w5_gnn_threat`	0.00	GNN Threat

Tabla 7.1: Pesos por defecto. El GNN está desactivado salvo que el usuario provea modelo y opte-in.

Los pesos se renormalizan automáticamente si no suman 1. El usuario puede sobreescribirlos por sesión vía `POST /enable_anomaly_scoring` o `POST /compute_scores`.

7.2. Componente 1: Entity Rarity

Mide qué tan infrecuentes son las entidades del path individualmente.

$$ER(v) = 1 - \frac{\ln(\text{freq}(v))}{\ln(\text{max_freq})} \qquad C_1(p) = \frac{1}{|p|} \sum_{v \in p} ER(v)$$

Donde $\text{freq}(v)$ es el conteo de apariciones del nodo v en eventos ingresados, y max_freq el máximo global. Un nodo que aparece una vez tiene $ER \approx 1$; el nodo más visto tiene $ER = 0$.

Uso típico: captura *qué tan raro es ver a estos actores del todo*. Sirve para priorizar paths donde participan cuentas o hosts rara vez observados.

7.3. Componente 2: Edge Rarity

Mide qué tan infrecuentes son las transiciones observadas.

$$EdgeR(s,d) = 1 - \frac{\ln(\text{freq}(s,d))}{\ln(\text{max_pair_freq})}$$

$C_2(p)$ se calcula como promedio sobre todas las aristas del path.

Uso típico: captura *qué tan inusual es esta secuencia concreta*. Un usuario que rutinariamente se autentica en un host tiene ER bajo para ambos nodos por separado, pero si un día ese usuario escribe en un directorio sensible al que nunca había ido, el edge rarity se dispara.

7.4. Componente 3: Neighborhood Concentration

Promedio sobre los nodos del path de su concentración de vecindario (precomputada al finalizar el grafo en `GraphHunter::finalize`).

$$NC(v) = \frac{\text{máx}_t \, |\{u : (v,u) \in E, \tau(u) = t\}|}{|\text{neigh}(v)|}$$

Donde $\tau(u)$ es el tipo de la entidad u. Un nodo cuyos vecinos son de tipos muy diversos tiene NC bajo; uno cuyos vecinos son casi todos del mismo tipo tiene NC alto (centralidad estructural).

Uso típico: *penaliza* paths que atraviesan hubs naturales (ej. un DC con 5000 usuarios conectados) porque su alto score en otras dimensiones se explica por topología, no por anomalía real.

7.5. Componente 4: Temporal Novelty

Qué tan recientemente apareció la entidad en la ventana observada.

$$TN(v) = \frac{\tau_{\text{first}}(v) - \tau_{\text{mín}}}{\tau_{\text{máx}} - \tau_{\text{mín}}} \qquad C_4(p) = \frac{1}{|p|} \sum_{v \in p} TN(v)$$

$TN(v) \approx 0$: el nodo existe desde el inicio de la ventana. $TN(v) \approx 1$: el nodo apareció al final.

Uso típico: separa *centralidad estructural* de *novedad genuina*. Un DC corporativo tiene $TN = 0$ (siempre estuvo); una IP externa recién observada tiene $TN \approx 1$.

7.6. Componente 5: GNN Threat

Score de amenaza por path según clasificador GNN entrenado (ver capítulo de Arquitectura, Capa Core). Cuando no hay modelo cargado, devuelve 0.0 para todo path — caso común en entornos limpios.

Clases de amenaza soportadas (`ThreatClass`): `Benign`, `Exfiltration`, `C2Beacon`, `LateralMovement`, `PrivilegeEscalation`. La probabilidad del argmax se usa como score. Peso por defecto 0.0 (opt-in).

7.7. Explicación

El endpoint `GET /explain_score` y la tool `scoring.explain_score` devuelven el desglose:

```
{
  "path_id": "...",
  "composite": 0.73,
  "components": {
    "entity_rarity":              0.82,
    "edge_rarity":                0.91,
    "neighborhood_concentration": 0.45,
```

```
    "temporal_novelty":           0.64,
    "gnn_threat":                 0.00
  },
  "weights": { "w1": 0.25, "w2": 0.30, "w3": 0.25, "w4": 0.20, "w5": 0.0 },
  "narrative": "entity rare (0.82), transition rare (0.91), ..."
}
```

`narrative` es texto humano generado en `core/graph-engine/src/analytics.rs::build_narrative`. No substituye el desglose numérico; lo acompaña.

7.8. Invariantes

- Cada $C_i(p) \in [0, 1]$.
- $S(p) \in [0, 1]$ estrictamente tras renormalización.
- Path vacío devuelve todos los componentes en 0.0 sin panic.
- Si el grafo no está finalizado, el scorer cortocircuita a 0.0 (precondición, no error).
- Todos los pesos en $0 \Rightarrow S(p) = 0$ para cualquier p.

Capítulo 8

Variables de entorno

8.1. Convención

Todas las variables del producto tienen prefijo `GRAPHHUNTER_`. La precedencia es:

1. Variable del proceso (seteada por el OS o por el shell).

2. `.env` en el working dir del Tauri daemon.

3. Default hardcodeado en código.

Un proceso arrancado con una variable explícita ignora `.env`. Esto es deliberado: en producción `.env` no existe y las variables vienen del supervisor.

8.2. Daemon (Tauri app)

Variable	Default	Propósito
`GRAPHHUNTER_API_TOKEN`	autogenerado	Bearer token para la API HTTP. Impreso al arrancar; copiarlo al cliente MCP.
`GRAPHHUNTER_API_PORT`	`37891`	Puerto TCP.
`GRAPHHUNTER_API_HOST`	`127.0.0.1`	Host de bind. Usar `0.0.0.0` sólo en entornos confiables.
`GRAPHHUNTER_CATALOG_DIR`	`\$HOME/.graphhunter/catalog`	Directorio con YAMLs de catálogo custom.
`GRAPHHUNTER_LOCAL_MODEL_PATH`	(resolver)	Override manual de la ruta al modelo GGUF para el backend *local-llm*. Por defecto los binarios Tauri/CLI invocan `graph\_hunter\_local\_llm::resolve\_model\` que busca el `.gguf` en (1) este *env var* si está seteado, (2) `<exe>/resources/models/` y `<exe>/models/` (ubicaciones *bundled* por `tauri build` y el instalador del CLI), (3) un *walk-up* desde `cwd` buscando `target/models/`. Si ninguna resuelve, se cae al `MockBackend`.
`GRAPHHUNTER_IOC_PUBLISH_ENABLED`	`0`	Habilita `POST /publish_iocs_to_sentinel`.
`GRAPHHUNTER_TICKET_PUBLISH_ENABLED`	`0`	Habilita `POST /file_ticket`.

Tabla 8.1: Variables consumidas por el daemon.

8.3. Cliente MCP

Variable	Default	Propósito
`GRAPHHUNTER_API_URL`	`http://127.0.0.1:37891`	URL del daemon.
`GRAPHHUNTER_API_TOKEN`	(obligatorio)	Mismo token que el daemon imprime.
`GRAPHHUNTER_MCP_DEBUG`	`0`	Log verboso.
`GRAPHHUNTER_MCP_ALLOW_NO_TOKEN`	`0`	*Sólo dev*: permite arrancar sin token.
`GRAPHHUNTER_CONTRACT_TEST`	`0`	Habilita suite de tests contractuales.

Tabla 8.2: Variables consumidas por el servidor MCP.

8.4. Integraciones externas

Consumidas por `POST /file_ticket`:

- **Jira**: `JIRA_BASE_URL`, `JIRA_EMAIL`, `JIRA_API_TOKEN`, `JIRA_PROJECT_KEY`, `JIRA_ISSUE_TYPE` (default `Task`).
- **ServiceNow**: `SERVICENOW_INSTANCE`, `SERVICENOW_USERNAME`, `SERVICENOW_PASSWORD`, `SERVICENOW_TABL` (default `incident`).

Consumidas por `POST /publish_iocs_to_sentinel`: credenciales de Azure (`AZURE_TENANT_ID`, `AZURE_CLIENT_ID`, `AZURE_CLIENT_SECRET`) y el workspace ID de Sentinel (`SENTINEL_WORKSPACE_ID`).

8.5. Feature flags

Estas dos flags son las que cambian superficie expuesta:

- `GRAPHHUNTER_IOC_PUBLISH_ENABLED=1`: sin esto, `/publish_iocs_to_sentinel` responde 403.

- `GRAPHHUNTER_TICKET_PUBLISH_ENABLED=1`: análogo para `/file_ticket`.

Cada tool MCP con efectos externos también pide `confirm:true` por llamada, o `dry_run:true` para previsualizar, sobre los env vars anteriores. Doble barrera.

8.6. Flags de *performance*

Las siguientes flags activan rutas de *matcher* optimizadas incorporadas en el *M-track* de abril 2026. Todas son *opt-in* y compatibles con la API existente: cuando están desactivadas el comportamiento es el del *path* legacy.

Variable	Default	Propósito
`GRAPHHUNTER_K4_LFTJ`	`0`	Habilita el detector *name marker* S3 que rutea hipótesis con *shape K_4* hacia el *enumerador* LFTJ `search\_temporal\_pattern\_k4\_lftj`.
`GRAPHHUNTER_LFTJ_PLANNER`	`0`	Habilita el detector *estructural* M1 (6 pasos homogéneos sin predicados) que rutea a LFTJ-K4 sin requerir *name marker*. Ortogonal a `K4\_LFTJ`; ambos pueden coexistir.
`GRAPHHUNTER_YANNAKAKIS`	`0`	L1: habilita el *matcher* de cadenas *output-sensitive* (semi-join reducer de dos pasadas + *path enumerator*). Aplica a toda hipótesis α-acíclica — las cadenas del DSL actual lo son. Cae al DFS legacy cuando el *planner* detecta que la hipótesis tiene un ciclo (vía *shared bindings*) o cuando el *flag* está apagado. Default-on en los binarios Tauri/CLI; override con `GRAPHHUNTER_YANNAKAKIS=0`.
`GRAPHHUNTER_AMAC`	`0`	Activa el *DFS interleaved* estilo AMAC con $K = 8$ contextos en vuelo por *worker* de Rayon, pensado para *hide* latencia de DRAM en patrones ≥ 3 saltos.
`GRAPHHUNTER_SEMIJOIN`	`0`	Activa la pasada *semi-join* bidireccional D2-lite que poda *candidate sets* antes de entrar al DFS.
`GRAPHHUNTER_HUGEPAGES`	`0`	Sólo Linux: invoca `madvise(MADV\_HUGEPAGE)` sobre regiones `Mmap` para reducir *TLB stalls* en grafos $> L2$.
`GRAPHHUNTER_LLM_INGEST`	`1`	Habilita el *dispatcher LLM-as-compiler* en el *hot-path* de *ingest*. Cuando la heurística produce un `FieldConfig` de baja confianza (cero *nodes* canónicos), el *dispatcher* invoca el *compiler constrained-decode* (Phi-3 + gramática VRL-subset) y muestra un *banner* de confirmación con el *diff* antes de insertar *triples*. Default-on en los binarios Tauri/CLI siempre que `graph\_hunter\_local\_llm::resolve\_model\_path` encuentre un `.gguf` en disco; `=0` desactiva el *dispatcher* y mantiene el comportamiento heurístico de las versiones previas.
`GRAPHHUNTER_MEM_WARN`	`0.75`	Heavy-logs: *ratio* $\in (0, 1]$ sobre el cual la *pildora Mem:* del *toolbar* pasa a *amber*. Usado por la UI para señalar presión de memoria moderada.
`GRAPHHUNTER_MEM_CRITICAL`	`0.90`	Heavy-logs: análogo a `MEM\_WARN` pero para el estado *red*. Por encima de este *ratio* la *pildora* aparece en rojo y la sesión está cerca del límite de la host RAM.
`GRAPHHUNTER_INDEX_HEAP_BUDGET`	`1073741824`	Heavy-logs: *budget* en *bytes* sobre el cual los *auxiliary indexes* (NLF, k-hop reachability) se *spillan* a un *temp file mmap-backed* en lugar de quedar en *heap*. Default = 1 GiB. Sobre un EVTX de 28 GB cada *array* temporal de `KHopReach` pesa $\sim$2.16 GB y por lo tanto se *spilla* automáticamente, man-

Capability de simetría *nauty*. El módulo `core/graph-engine/src/nauty\_symmetry.rs` expone una constante `HAS\_NAUTY: bool` que vale `true` sólo cuando el *crate* se compiló con `--features symmetry-nauty` sobre un *target* `cfg(unix)`. En Windows o sin la *feature* la *capability* es `false` y el módulo retorna *constraints* vacíos como *soundness-preserving no-op*. No es una variable de entorno — es una decisión de *build*, descrita en `core/graph-engine/Cargo.toml`.

8.7. Fuente canónica

El chequeo de env vars ocurre en:

- `apps/tauri/src-tauri/src/lib.rs`: token, port.
- `apps/tauri/src-tauri/src/http/mod.rs`: host.
- `core/graph-engine/src/catalog/loader.rs`: catalog dir.
- `platform/local-llm/src/lib.rs`: model path.
- `graph-hunter-mcp/src/index.ts`: cliente MCP.

Agregar una variable nueva obliga a actualizar este capítulo y `.env.example`.

Capítulo 9

Errores y códigos

9.1. HTTP API

El daemon sigue códigos HTTP estándar. Cada error trae un body JSON:

```
{
  "error":   "dsl_parse",
  "message": "unexpected token at line 1, col 14",
  "details": { ... }
}
```

Código	`error`	Causa típica
400	`bad_request`	DTO malformado, tipo incorrecto.
401	`unauthorized`	Header `Authorization` ausente o token inválido.
403	`forbidden`	Feature flag desactivado.
404	`not_found`	Nodo, hunt, nota o sesión inexistente.
409	`conflict`	Ingesta en curso, sesión bloqueada.
422	`dsl_parse`	DSL inválido.
422	`schema_drift`	Batch viola tolerancia de drift.
422	`invariant_failed`	Grafo hipotético rompe invariantes.
500	`internal`	Error no clasificado. Revisar log del daemon.

Tabla 9.1: Códigos HTTP emitidos por la API.

9.2. MCP tools

Cada tool MCP propaga el error de la API subyacente como excepción tipada. El cliente recibe:

```
{
  "isError": true,
  "content": [{
    "type": "text",
    "text": "MCP error: unauthorized (401) - GRAPHHUNTER_API_TOKEN missing
       or invalid"
  }]
}
```

Errores específicos de tools (antes de llegar al API):

- **Zod validation**: input no cumple `inputSchema` de la tool. Mensaje incluye el path Zod fallido.

- **Timeout**: 30s default (lectura), 300s (ingest / hunt). Tool devuelve `timeout`.
- **Connection refused**: daemon no corre o URL mal configurada. Mensaje incluye `GRAPHHUNTER_API_URL` efectivo.
- **HTML en lugar de JSON**: otro servicio ocupa el puerto. Error `not_graphhunter`.

9.3. DSL parser

Errores emitidos por `POST /parse_dsl` y por `POST /run_hunt`:

`error`	Significado
`unexpected_token`	Token no esperado en esa posición.
`unknown_entity_type`	Tipo no registrado en el `ExtensionRegistry`.
`unknown_relation`	Ídem, relación.
`unknown_predicate`	Predicado no implementado.
`invalid_duration`	Formato de `window` incorrecto.
`empty_hypothesis`	La hipótesis no tiene steps.

Todos los errores DSL traen `line` y `column`.

9.4. C ABI (histórica)

Nota 2026-05-07: el motor `libgraphmatch` fue eliminado; los contratos siguientes describen el comportamiento de la ABI mientras existió (cap. 6):

- `gm_matcher_run(m, g)`: devuelve `NULL` si `m` mal configurado o `g` no finalizado.
- `gm_results_get_*`: índice fuera de rango devuelve `NULL` y `*out_len = 0`.
- Mismatch de `gm_abi_version`: el cliente Rust aborta con panic (el binario no arranca).
- Violación de invariantes (doble free, uso después de free): UB. El wrapper Rust `GmGraph` previene esto por typestate y RAII.

9.5. Ingesta

La ingesta tiene errores granulares (ver también Dead Letter Queue, §2):

`error`	Significado
`parser_not_found`	Ningún parser reconoce la muestra.
`mapping_missing_time`	`FieldConfig` sin rol `TIME` obligatorio.
`time_parse_failed`	Ningún formato de timestamp matchea.
`dlq_enqueued`	Evento parseable pero violaba invariante; guardado en DLQ.
`session_locked`	Otra ingesta está corriendo en esa sesión.

Los eventos en DLQ se listan con `POST /list_dead_letters` y se reintentan con `POST /reingest_dead_letter`.

9.6. Convención de log

Todos los errores relevantes se loguean en el daemon con nivel `error` o `warn`. Correlación vía `request_id` inyectado en cada request HTTP (header `X-Request-Id`). Revisar `app.log` para detalle.

Apéndice A

Historial de cambios de superficie

A.1. Política de versiones

HTTP API y **MCP tools** tienen ciclo independiente. Cada una sigue semver:

- **Mayor**: cambio incompatible (remoción de endpoint/tool, cambio de shape).
- **Menor**: agregado compatible (nuevo endpoint, campo opcional).
- **Patch**: corrección sin cambio de shape.

El DSL tiene su propio versionado (`v1` estable, `v2` experimental detrás de feature flag). La C ABI tiene versión entera monotónica (`gm_abi_version()`).

A.2. v1.0 — baseline estable

Corresponde al cierre del rediseño shearing-layers. Lo que entra a v1.0:
HTTP API: 62 endpoints de producto + `/` + `/metrics`. Todos los listados en cap. 2 son `stable` salvo:

- `POST /publish_iocs_to_sentinel` — `experimental` (gated por feature flag).
- `POST /file_ticket` — `experimental` (gated por feature flag).
- `POST /kql` — `experimental` (depende de credenciales externas).

MCP tools: 55 tools en 10 dominios. Todas `stable` salvo:

- `integrations.publish_iocs_to_sentinel`.
- `integrations.file_ticket`.
- `scoring.enable_anomaly_scoring` cuando `w5_gnn_threat > 0`.

DSL: gramática v1 (ver cap. 4).
C ABI: `gm_abi_version() = 1`.

A.3. Compromisos v1

- Un endpoint `stable` no se elimina ni cambia de método/path dentro de v1. Se puede agregar campo opcional nuevo al request o response.
- Una tool MCP `stable` no cambia su `inputSchema` de forma incompatible dentro de v1.
- Una hipótesis DSL válida en v1 sigue parseando igual en toda la rama v1.
- `gm_abi_version()` cambia si y sólo si hay cambio incompatible en la C ABI.

A.4. Qué queda fuera de v1

- **Agentic slow-lane** — `agentic_*` endpoints y tools. Interfaces funcionan pero se considera `experimental` mientras no se cierre el backend de LLM local.
- **GNN v2** — model gate y `scoring.explain_score.threat_class` sólo son relevantes si el usuario provee modelo ONNX. Sin modelo, comportamiento es estable (devuelve 0.0).
- **DSL v2** — operadores temporales explícitos (`before`, `within`) detrás del feature `dsl-v2-experimental`.

A.5. Tags git de release

- `platform/api-v1.0.0` — primer tag estable de la API HTTP.
- `platform/mcp-v1.0.0` — primer tag estable del servidor MCP.
- `libgraphmatch-v1.0.0` — C ABI congelada en `gm_abi_version() = 1`. *Removido el 2026-05-07*: el motor C++ fue eliminado tras la migración al DFS puro-Rust.

El workspace raíz sigue semver de producto (versión del desktop app), independiente de estos.

A.6. Fuente canónica

El listado *vivo* de endpoints es `GET /v1/schema`. Si hay discrepancia entre este documento y la respuesta del daemon, el daemon gana; este documento se actualiza en el siguiente release.

Para tools MCP, el listado canónico es `GET /v1/mcp/schema` (emitido por el servidor MCP) o `platform/mcp/src/registry.ts` en el código fuente.